F. Deadlock + Banker's Algorithm (OSNo07) 🔥

> **📖 รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** Deadlock (การติดตาย) = กลุ่ม process รอทรัพยากรกันเป็นวงกลม ไม่มีใครได้ทำงานต่ออีกเลย · Banker's Algorithm = วิธี "เช็กก่อนให้" ว่าจัดสรรแล้วระบบยังปลอดภัยไหม
>
> **ความสำคัญกับข้อสอบ:** เงื่อนไข 4 ข้อ (ต้องครบพร้อมกัน), อ่าน Resource-Allocation Graph (กราฟแสดงการจัดสรรทรัพยากร), และไล่ Safety Algorithm (อัลกอริทึม (ขั้นตอนวิธี)) หา Safe Sequence (ลำดับการรันที่ปลอดภัย) — เป็นแนวคำนวณหลักของ Final
>
> **ใช้จริงในคอมพิวเตอร์:** ฐานข้อมูลล็อกแถวข้อมูล 2 ธุรกรรมรอกัน, เปิดไฟล์ Excel 2 โปรแกรมล็อกกัน, รถติด 4 แยกไม่มีใครยอมถอย — ระบบจริงเลือก "เมิน + ให้คนรีสตาร์ทเอง" เหมือนที่ OS ทำ

F1. Deadlock คืออะไร + ตัวอย่างรากฐาน

Deadlock = process ≥ 2 ตัว**รอกันไม่สิ้นสุด** เพราะเหตุการณ์ที่จะปลุกนั้นอยู่ในมือกลุ่มผู้รอเอง

```
/* ตัวอย่างโค้ดจากสไลด์ p.4 — S, Q เป็น binary semaphore เริ่ม 1 ทั้งคู่ */
/* P0 */                  /* P1 */
wait(S);   ← S=0 ยึด S    wait(Q);   ← Q=0 ยึด Q
wait(Q);   รอ Q (P1 ถือ)   wait(S);   รอ S (P0 ถือ) → ตายล็อกทั้งคู่
...
signal(S); signal(Q);     signal(Q); signal(S);
```

- **Starvation** = indefinite blocking — process อาจไม่เคยถูกหยิบออกจากคิว semaphore เลย (ต่างจาก deadlock ที่รอกันเป็นวง)
- สไลด์ p.3 สะพานแคบ: รถ = process, ช่วงถนน = resource, แก้โดยให้คันหนึ่ง**ถอย (preempt + rollback)**
- **OS ส่วนใหญ่ (รวม UNIX) ไม่ป้องกัน/ไม่จัดการ deadlock เลย** — ปล่อยให้โปรแกรมเมอร์รับผิดชอบ

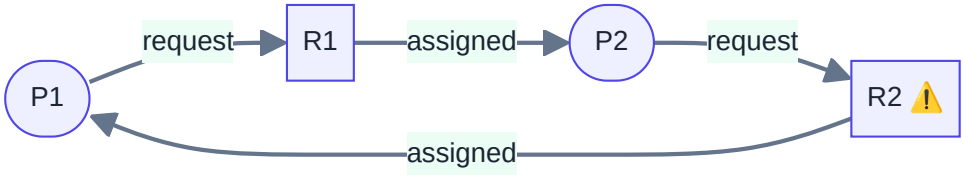
F2. เงื่อนไข 4 ข้อ — ต้องเกิด "พร้อมกัน" จึงเกิด deadlock 🔥

เงื่อนไข	ความหมาย	Prevention ยังไง
1. Mutual Exclusion	ทรัพยากร 1 ชิ้นมีได้ 1 เจ้าของ/เวลา	ทำให้แชร์ได้ — ทำไม่ได้กับทุกชนิด
2. Hold and Wait (ถือของเก่า รอของใหม่)	ถืออย่างน้อย 1 ชิ้น + รอชิ้นอื่นที่คนอื่นถือ	ขอให้**ครบทุกชิ้นก่อนเริ่มรัน** หรือขอเมื่อไม่ถืออะไรเลย → utilization ต่ำ + starvation ได้
3. No Preemption (ยึดคืนกลางคันไม่ได้)	ยึดคืนกลางคันไม่ได้ ปล่อยเมื่อเจ้าของใจดีเท่านั้น	ขอเพิ่มไม่ได้ = ถูกยึดของที่ถือทั้งหมด ไปให้คนรอ แล้วค่อยเริ่มใหม่ → ยุ่งยาก
4. Circular Wait (รอกันเป็นวงกลม)	P0 รอ P1, P1 รอ P2, ... วนกลับมา P0	ให้ทุกทรัพยากรมี**เลขประจำตัว** ขอเรียง**จากน้อยไปมากเท่านั้น** เช่น Disk=4, Printer=11 → ต้องได้ 4 ก่อน 11 ห้ามถือ 11 แล้วขอ 4

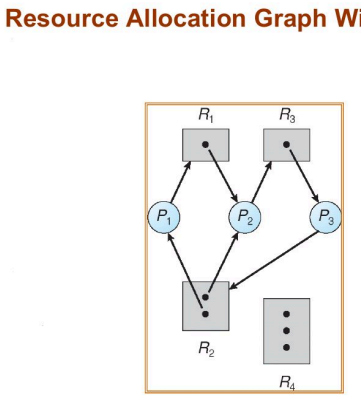
> **🎯 จำแน่น**
>
> ต้องครบ **4 ข้อพร้อมกัน** → ตัดข้อเดียวก็ไม่เกิด · Prevention ทั้งหมด "ทฤษฎีดี แต่ปฏิบัติเสียประสิทธิภาพ" อาจารย์สรุปว่าไม่คุ้มใช้

F3. Resource-Allocation Graph (RAG)

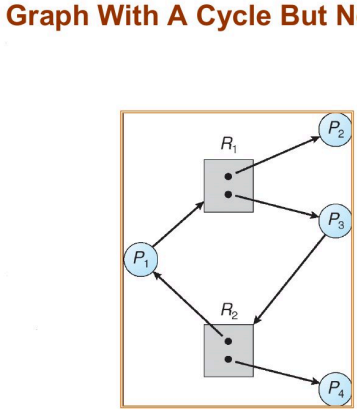
- **วงกลม** = Process P_i · **สี่เหลี่ยม** = Resource (ทรัพยากร) type R_j (จุดในสี่เหลี่ยม = จำนวน instance)
- **Request edge** $P_i \rightarrow R_j$: "ขอ" · **Assignment edge** $R_j \rightarrow P_i$: "ได้ถืออยู่"



ไม่มี cycle (วงจร (วนกลับมาที่เดิม)) → ไม่เกิด deadlock แน่ (100%)
มี cycle → single instance/type = deadlock แน่ · multiple instances = "อาจเกิด" เท่านั้น
slide p.11: มี cycle แต่ R2 มีหลายชิ้น ชิ้นที่เหลือ (ถือโดย P4 นอกวง) มาช่วยได้ → ไม่ deadlock



สไลด์ p.10 RAG มี deadlock



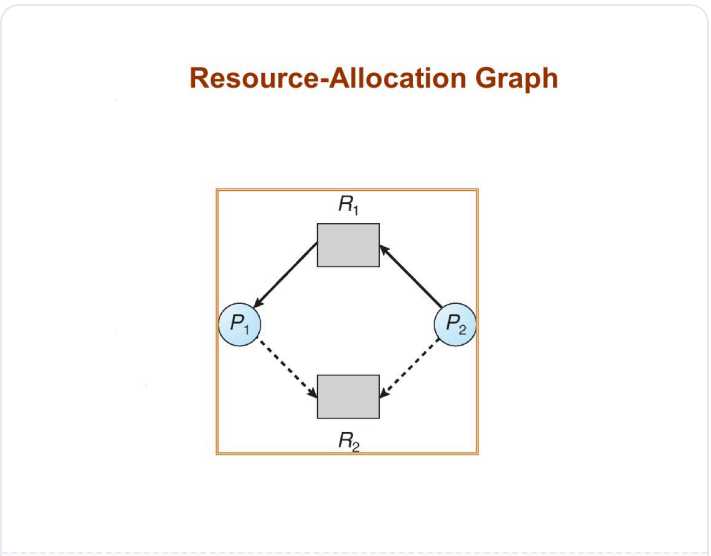
สไลด์ p.11 มี cycle แต่ไม่ deadlock (multiple instance)

F4. กลยุทธ์จัดการ Deadlock 4 แนวทาง

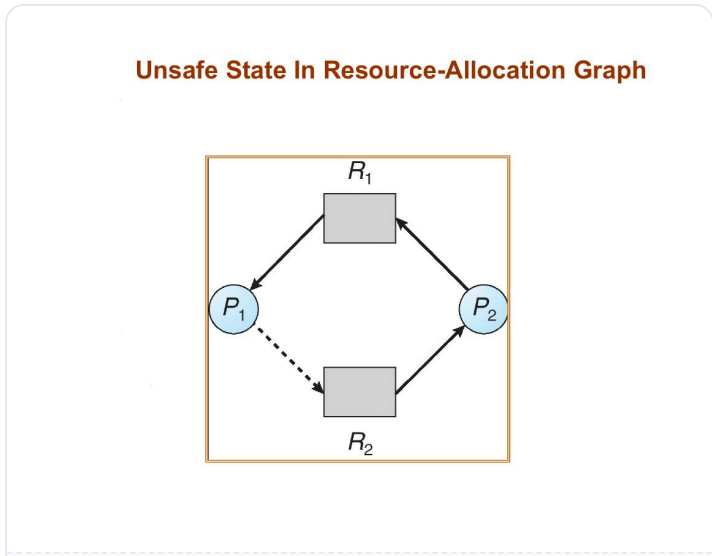
1. **Prevention** — ออกแบบให้เงื่อนไขข้อใดข้อหนึ่งเกิดไม่ได้เลย (ตาราง F2)
2. **Avoidance** — process แจ้ง **Max demand ล่วงหน้า** OS เช็คทุกครั้งก่อนจัดสรรว่ายัง safe → Banker (อัลกอริทึมนายธนาคาร)'s / claim edge
3. **Detection + Recovery** — ปล่อยเกิด สแกนหา แล้วแก้
4. **เมิน (Ostrich)** — ที่ OS จริงใช้กัน

F4.1 Avoidance แบบ single instance — Claim Edge (เส้นประ)

- **Claim edge** $P_i \dashrightarrow R_j$ (เส้นประ) = "อาจขอในอนาคต" · ขอจริง → กลายเป็น request edge · ได้รับ → assignment edge · ปล่อย → กลับเป็น claim edge
- **ให้ทรัพยากรได้เมื่อ:** แปลง request เป็น assignment แล้ว**เกิด cycle ไม่ได้** (มี cycle = unsafe → ไม่ให้ ให้รอ)
- ข้อเสีย: สแกนหา cycle ทุกครั้งที่ขอ = $O(n^2)$ แพง



สไลด์ p.20 Claim edges (เส้นประ)



สไลด์ p.21 Unsafe state: ถ้าให้ P2 ใช้ R2 จะเกิดวง → ไม่ให้

F5. Banker's Algorithm (multiple instances) 🔥🔥

ข้อมูลที่ OS ถือ (n = process, m = resource type): `Available[m]` · `Max[n×m]` · `Allocation[n×m]` · **Need = Max − Allocation**

```
Need[i,j] = Max[i,j] − Allocation[i,j]
```

หัวใจของทุกโจทย์ Banker's — คิด Need ให้ครบทุกช่องก่อนเสมอ

F5.1 Safety Algorithm (ตรวจว่า state ปลอดภัยไหม)

1. Work = Available · Finish[i] = false ทุกตัว
2. หา i ที่ **Finish[i] = false และ $Need_i \le Work$** (เทียบทีละช่องทุกชนิด) — ไม่มีใครผ่าน → ไปขั้น 4
3. สมมติ i รันจนจบแล้วคืนของ: **Work = Work + $Allocation_i$**, Finish[i] = true → กลับขั้น 2
4. ถ้า Finish = true **ทุกตัว** → **SAFE** (มี safe sequence) · ติดตรงไหน = UNSAFE

F5.2 โจทย์เต็มจากสไลด์ p.26–27 (5 process, A=10 B=5 C=7, Available = 3,3,2)

Process	Allocation			Max			Need = Max − Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

ไล่ Safety (Work เริ่ม = 3,3,2):

ขั้น	เลือกใคร (เหตุผล)	Work หลังคืนของ
1	P1 — Need (1,2,2) ≤ (3,3,2) ✓	(3,3,2)+(2,0,0) = **(5,3,2)**
2	P3 — Need (0,1,1) ≤ (5,3,2) ✓	(5,3,2)+(2,1,1) = **(7,4,3)**
3	P4 — Need (4,3,1) ≤ (7,4,3) ✓	(7,4,3)+(0,0,2) = **(7,4,5)**
4	P2 — Need (6,0,0) ≤ (7,4,5) ✓	(7,4,5)+(3,0,2) = **(10,4,7)**
5	P0 — Need (7,4,3) ≤ (10,4,7) ✓	(10,4,7)+(0,1,0) = (10,5,7) ครบทุกตัว ✅

`Safe sequence = <P1, P3, P4, P2, P0> → SAFE STATE`

มีได้หลายคำตอบ — ตอบลำดับไหนที่ไล่ได้จริงก็ถูก (สไลด์ให้ <P1,P3,P4,P2,P0>)

F5.3 Resource-Request Algorithm — P1 ขอ (1,0,2) ให้ได้ไหม?

1. เช็ค Request ≤ Need: (1,0,2) ≤ (1,2,2) ✓ (ไม่ขอเกินที่ประกาศ)
2. เช็ค Request ≤ Available: (1,0,2) ≤ (3,3,2) ✓ (ของพอ)
3. **สมมติจัดให้ (pretend):** Available = (2,3,0) · P1 Alloc = (3,0,2) · P1 Need = (0,2,0)
4. รัน Safety ใหม่ → ได้ sequence <P1, P3, P4, P0, P2> → ยัง SAFE ✅ → **ให้ไปได้**

คำขออื่นในสไลด์ p.28	ผล	เหตุผล
P4 ขอ (3,3,0)	**ไม่ให้**	Request > Available (3,3,0) > (2,3,0) — ของไม่พอ P4 ต้อง wait
P0 ขอ (0,2,0)	**ไม่ให้**	ผ่าน 2 เช็คแรก แต่ไล่ Safety แล้ว **UNSAFE** (มี deadlock เป็นไปได้) → ให้รอ คืนสถานะเดิม

> **🟢 วิธีทำในห้องสอบ (เร็ว)**
>
> 1) เติมตาราง Need = Max − Alloc ทุกช่อง 2) เขียน Work = Available กก.ลงข้าง ๆ 3) วนหา "Need ≤ Work" ใครผ่านให้ไปก่อน แล้ว **บวก Allocation แถวนั้นเข้า Work** 4) จนครบทุกตัว = safe · โจทย์ถาม "ขอได้ไหม" → เช็ค 2 ข้อแล้ว pretend แล้วไล่ safety ใหม่เสมอ

> **📌 ปัญหาของ Banker's (เคยออก)**
>
> ต้องประกาศ Max ล่วงหน้า / จำนวนทรัพยากรคงที่ / จำนวนงานคงที่ / overhead สูง / ใช้ทรัพยากรไม่คุ้มเพราะคิดแย่สุดเสมอ

F6. Detection (ตรวจจับ) & Recovery (กู้)

- **Single instance/type:** ยุบ RAG เป็น **Wait-For Graph (กราฟแสดงว่าใครรอใคร)** — ตัดโหนดทรัพยากรทิ้ง เหลือ $P_i \rightarrow P_j$ ("P_i รอ P_j") มี cycle = deadlock (สแกนทุกระยะ, แพง $O(n^2)$)
- **Multiple instances:** Detection Algorithm ทำงานเหมือน Safety แต่เทียบด้วย **Request** (ของที่ร้องขอค้างจริง) แทน Need · Finish เริ่มต้น: คนที่ Allocation ไม่เป็นศูนย์ = false

ตัวอย่าง Detection จากสไลด์ p.36–37 (A=7, B=2, C=6, Available = 0,0,0):

Process	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	2	0	0	2	0	2
P2	3	0	3	0	0	0
P3	2	1	1	1	0	0
P4	0	0	2	0	0	2

ไล่: Work=(0,0,0) → P0 (Request 000 ≤ 000) จบ → Work=(0,1,0) → P2 (000 ≤ 010) จบ → Work=(3,1,3) → P3 จบ → (5,2,4) → P1 จบ → (7,2,4) → P4 จบ → **sequence <P0,P2,P3,P1,P4> ครบ → ไม่มี deadlock**

แต่ถ้า P2 ขอเพิ่ม C หนึ่งชิ้น (Request P2 = 0,0,1): Work=(0,0,0) → P0 จบ → (0,1,0) → ใครผ่าน? P1(2,0,2)✗ P2(0,0,1)✗ P3(1,0,0)✗ P4(0,0,2)✗ → ตัน → **deadlock: P1, P2, P3, P4**

Recovery 2 แนวทาง

- **Process Termination:** จบทุกตัวที่ติด (เสียงานหมด) หรือจบทีละตัวจนวงแตก — เลือกจาก: priority, รันมานานแค่ไหน, ใช้ทรัพยากรอะไรไปแล้ว, ต้องการอีกเท่าไร, ต้องจบกี่ตัว, interactive/batch?
- **Resource Preemption (การยึดทรัพยากรกลางคัน):** เลือกเหยื่อ (victim) → **Rollback** ไป safe state → เริ่มใหม่ · ระวัง **Starvation** — ใส่ "จำนวน rollback ที่โดนมาแล้ว" ในตัวแปรต้นทุน